NETWORKS AND PROTOCOLS

Lab 05 - Wireshark/Scapy - ICMP

ICMP Protocol

Objective

The Internet Control Message Protocol (ICMP) plays a crucial role in supporting the Internet protocol suite by assisting IP with error handling and diagnostic functions. It is used by network devices, such as routers, to send error messages and operational updates, such as notifications about unavailable services or unreachable hosts or routers. Unlike transport protocols like TCP and UDP, ICMP is not designed for exchanging data between systems. Instead, it is primarily used for network diagnostics and troubleshooting tools, such as ping and traceroute, rather than by typical end-user applications.

Structure of an ICMP Message

An ICMP message is encapsulated in an IP packet, and its structure comprises :

ICMP Message

Type (1B)	Code (1B)	Checksum (2B)
Rest of Header (4B)		
ICMP data (variable)		

- **ICMP header :**
  - **Type (1 Byte):** Defines the nature of the message (for example, 'destination unreachable').
  - **Code (1 Byte):** Provides additional information about the type of message.
  - **Checksum (2 Bytes):** Checks the integrity of the ICMP message.
  - **Rest of Header (4 Bytes):** Four-byte field, contents vary based on the ICMP type and code.

- **Data:** Contains information specific to the type of ICMP message sent.

Here is a complete list of ICMP types for IPv4, including their main descriptions and uses. These types are defined in RFC standards, in particular RFC 792 (Basic ICMP) and subsequent updates.

Type	Name	Description
0	Echo Reply	Response to an echo request (used by `ping` to test connectivity).
1	Reserved	Reserved for future use.
2	Reserved	Reserved for future use.
3	Destination Unreachable	Indicates that the destination is unreachable (with various codes to specify the cause, e.g. network or host unreachable, port closed, etc.).
4	Source Quench (Deprecated)	Indicates that the network equipment is overloaded and requests a slowdown in the sending of packets.
5	Redirect Message	Tells the host to update its routing table to use another, more optimal path.
8	Echo Request	Sent by diagnostic tools like `ping` to check if a host is reachable and responsive.
9	Router Advertisement	Allows routers to announce their presence to hosts, providing information for host configuration.
10	Router Solicitation	Sent by hosts to request router advertisements.
11	Time Exceeded	Indicates that a packet's TTL (Time-to-Live) value has expired, often used in `traceroute` to identify intermediate routers.
12	Parameter Problem	Indicates an issue with the packet header, such as an invalid option or field.
13	Timestamp Request	Requests the current time from a network host.
14	Timestamp Reply	Responds to a timestamp request.
17	Address Mask Request	Sent by a host to determine its subnet mask.
18	Address Mask Reply	Provides the subnet mask in response to an address mask request.

The value of the code is related to the Type value of each ICMP message.

Here are the codes associated with the most frequently used ICMP types in IPv4, providing additional context or granularity for each type:

- **Destination Unreachable (Type 3):** Indicates that the destination is unreachable for a variety of reasons. The Code field provides specific details:

Code	Meaning
0	Network unreachable.
1	Host unreachable.
2	Protocol unreachable.
3	Port unreachable.
4	Fragmentation needed (and DF set).
5	Source route failed.
6	Destination network unknown.
7	Destination host unknown.
8	Source host isolated.
9	Communication with destination network administratively prohibited.
10	Communication with destination host administratively prohibited.
11	Network unreachable for TOS (Type of Service).
12	Host unreachable for TOS.
13	Communication administratively prohibited.
14	Host precedence violation.
15	Precedence cutoff in effect.

- **Time Exceeded (Type 11):** Used when a packet's lifetime (TTL) expires or reassembly time exceeds the limit:

Code	Meaning
0	TTL expired in transit.
1	Fragment reassembly time exceeded.

- **Redirect Message (Type 5):** Instructs a host to update its routing table to use a better route. The Code field indicates the type of redirect:

Code	Meaning
0	Redirect for the network.
1	Redirect for the host.
2	Redirect for TOS and network.
3	Redirect for TOS and host.

- **Parameter Problem (Type 12) :** Indicates an error in the IP header of the packet. The Code field specifies the type of error:

Code	Meaning
0	Pointer indicates the error.
1	Missing required option.
2	Bad length.

- **Echo Request (Type 8) and Echo Reply (Type 0):** Both are used by tools like ping. These types typically have Code 0, as no additional information is needed:
  - **Code 0:** Standard echo request or reply.
- **Timestamp Request (Type 13) and Timestamp Reply (Type 14):** Used for time synchronization. These types also generally have Code 0:
  - **Code 0:** Standard timestamp request or reply.

Exploring the ICMP protocol with Wireshark

Task 1: Capturing ICMP messages with ping

Step 1: Configure Wireshark

- Launch Wireshark.
- Select the active network interface.
- Apply a filter for ICMP : `icmp`. See figure 1

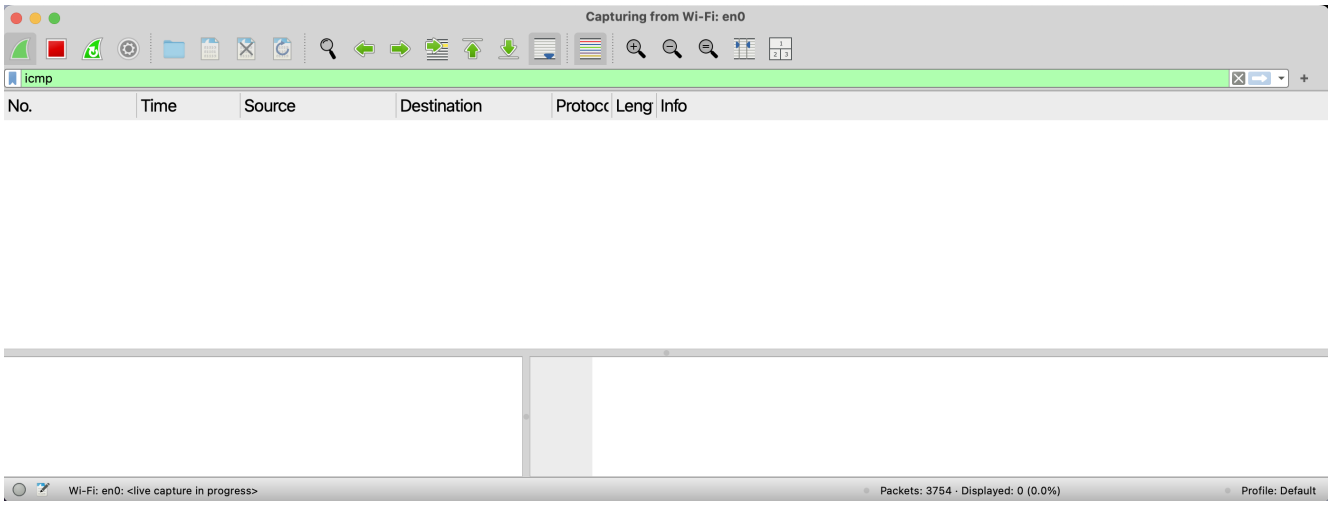

Figure 1: Wireshark with icmp filter

Step 2 : Run the ping command:

- Open a terminal and run the following command:

```
ping -c 4 8.8.8.8     #(Linux/macOS)
ping 8.8.8.8 -n 4     #(Windows)
```

- Server 8.8.8.8 (Google's public DNS) will respond with ICMP Echo Reply messages, Figure 2.

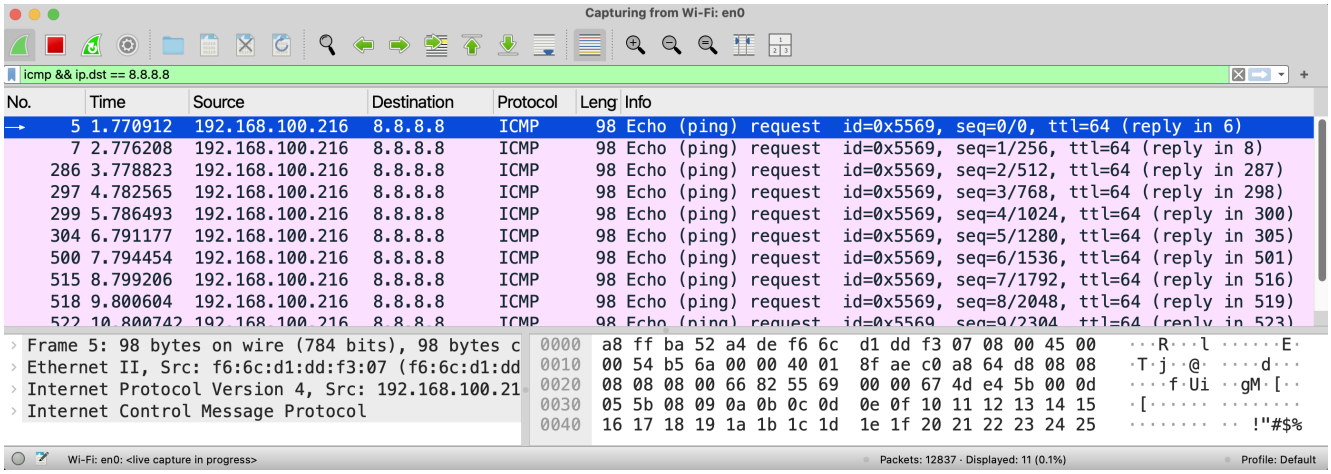

Figure 2: Ping result in Wireshark

Step 3: Analysing ICMP frames in Wireshark :

- Identify Echo Requests (Type 8, Code 0) sent by your machine, figure 3.
- Identify the Echo Reply (Type 0, Code 0) received in response, figure 4.
- Examine the important fields:
  - Identifier.
  - Sequence number.
  - Encapsulated data.

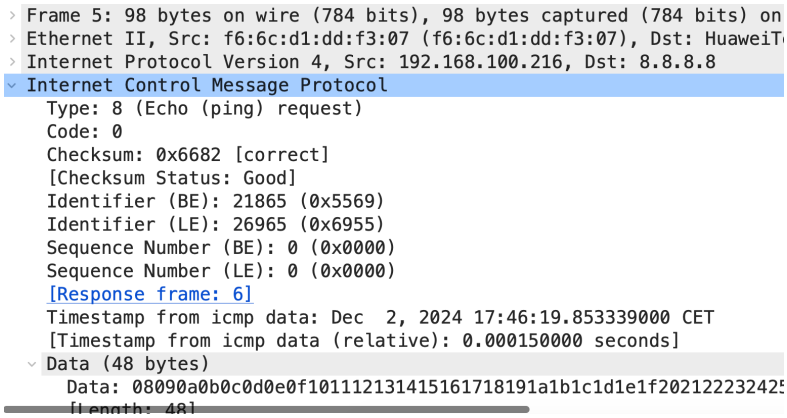

Figure 3: Echo request captured with Wireshart

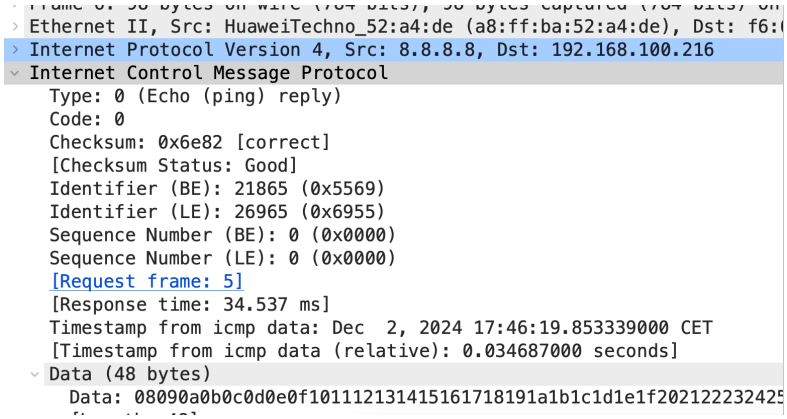

Figure 4: Echo reply captured with Wireshark

Task 2: Analysis of ICMP errors with an unreachable target

Step 1: Simulate an unreachable destination error :

- Run the `ping` command to a non-existent address :

```
ping -c 4 192.0.2.1    # Non routable IP address (Linux/macOS)
ping 192.0.2.1 -n 4    # (Windows)
```

- This address is part of a block reserved for documentation and will not be accessible.

Step 2: Observing ICMP messages in Wireshark :

- Filtering ICMP Destination Unreachable packets :

```
icmp.type == 3
```

- Identify the Code to understand the reason for the inaccessibility, figure 5:
  - Code 0: Network unreachable.
  - Code 1: Host unreachable.
  - Code 3: Port unreachable.

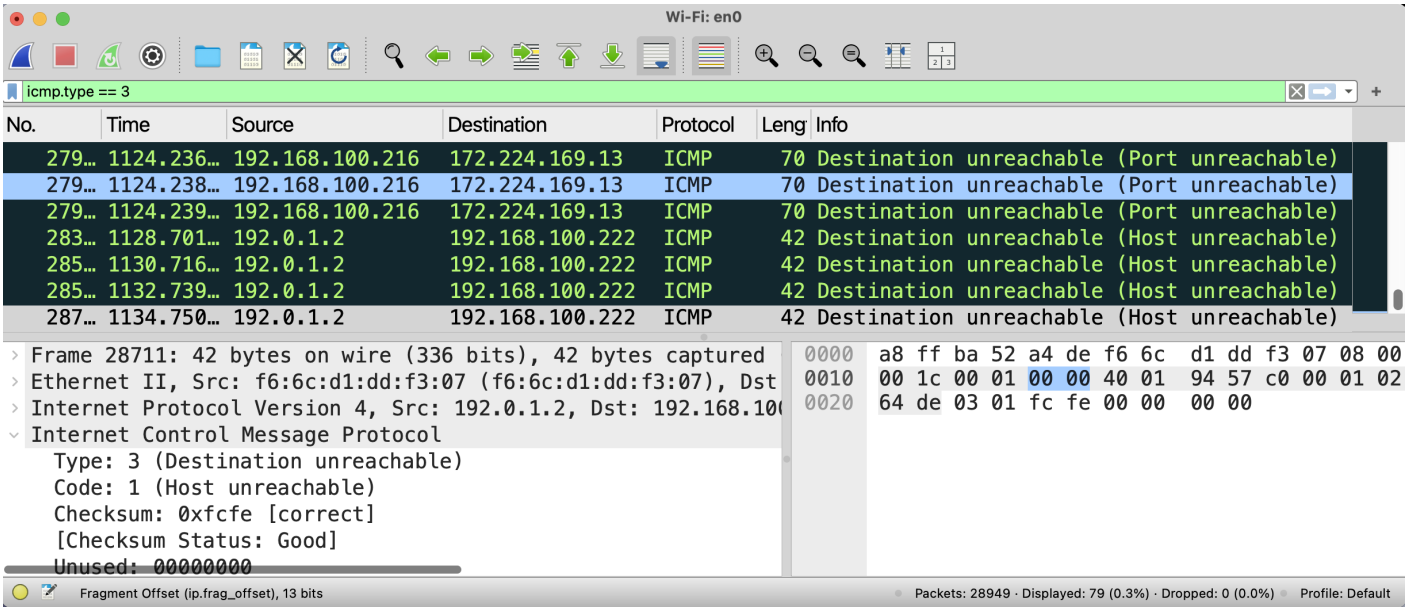

Figure 5: Host unreachable captured with Wireshart

Step 3: Study traceroute and Time Exceeded messages

- Run a `traceroute` command, figure 6:

```
traceroute 8.8.8.8 # (Linux/macOS)
tracert 8.8.8.8 # (Windows)
```

- Observing ICMP messages in Wireshark :
  - Identify Time Exceeded messages (**Type 11**, **Code 0**).

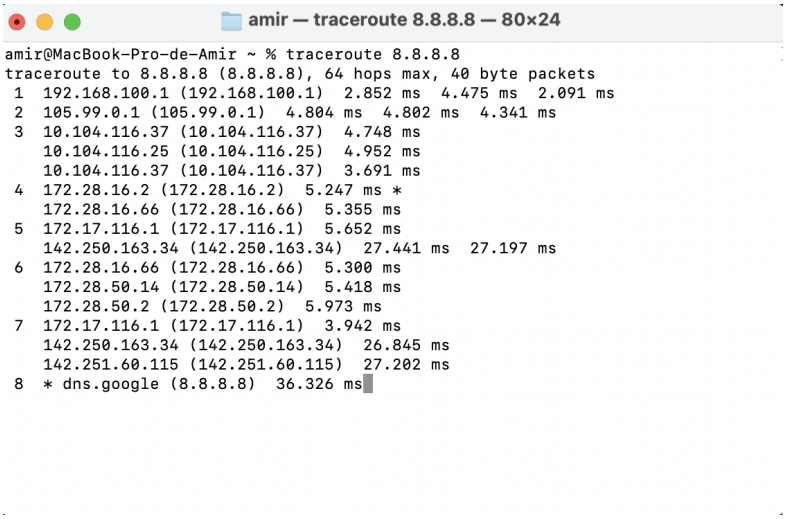

Figure 6: Traceroute 8.8.8.8

– Understanding their role :

  * They are sent by the intermediate routers when the TTL (Time-To-Live) of a packet reaches 0.

– Analyse the path taken by the packets to reach their destination, figure 7.

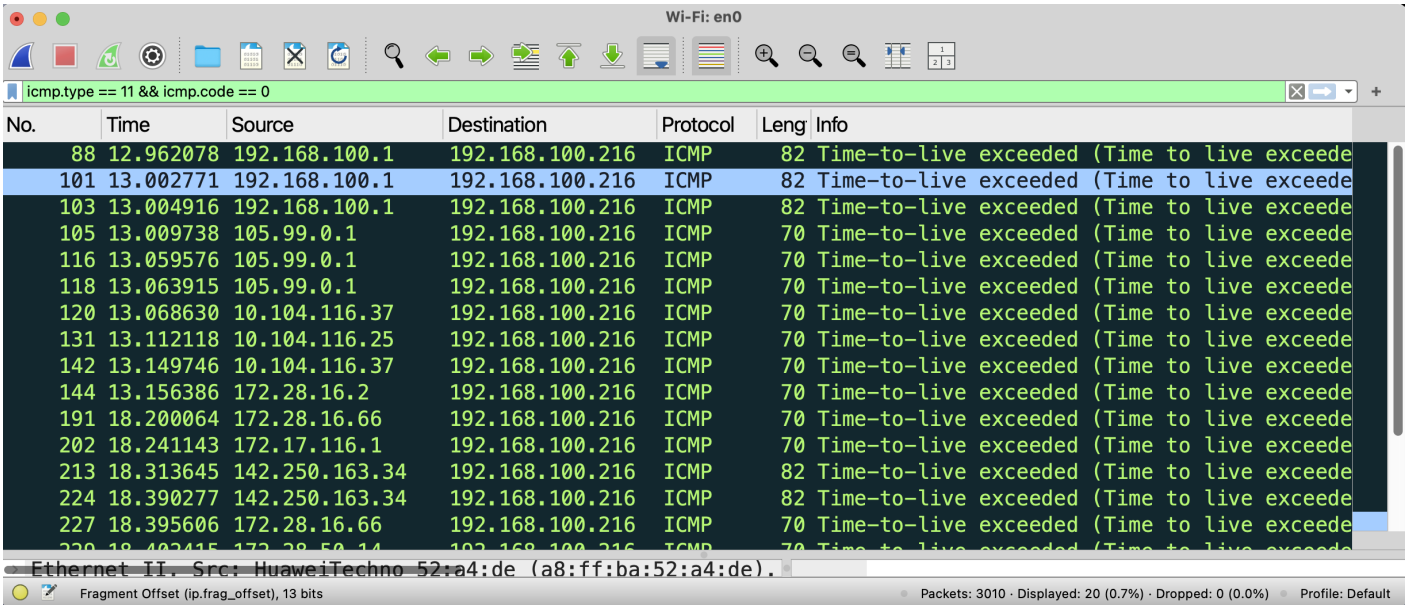

Figure 7: Traceroute packets captured by Wireshark

Task 3: Fragmentation examples
Step 1: Capture an example of fragmentation

- Apply a filter to capture only IP fragments :

```
ip.flags == 1 or ip.flags.mf == 1
```

- Generate IP fragments :

- Use the ping command to send large ICMP packets that require fragmentation:

```
ping -s 3000 8.8.8.8    # Linux/macOS (3000 Bytes length)
ping -l 3000 8.8.8.8    # Windows
```

- The `-s` or `-l` option specifies the size of the data sent. Adjust if necessary to force fragmentation.

- Analysing fragments in Wireshark :
  - Identify the fields in the fragment IP header, figure 8 :
    - **Flags:** Indicates whether a fragment is the last (`MF=0`) or whether other fragments remain (`MF=1`).
    - **Fragment Offset:** Shows the relative position of the fragment.
  - Reconstruct the original package by combining the fragments observed.

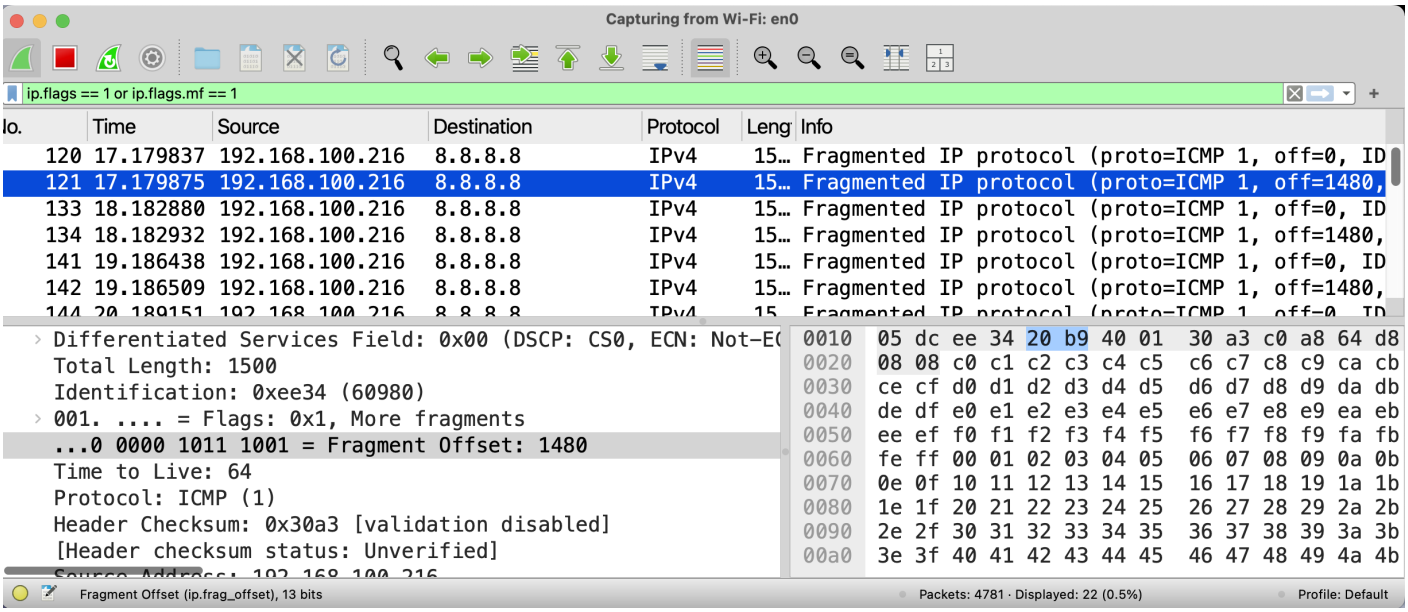

Figure 8: Fragments captured with Wireshark

Step 2: Generate a fragmentation error

- Sending a packet with the DF flag set :
  - Use the following command to send an ICMP packet with `DF` enabled and a size greater than the MTU:

```
ping -s 2000 -D 8.8.8.8    # Linux/macOS
ping 8.8.8.8 -l 2000 -f    # Windows
```

  - Or a ping with a size less than the MTU:

```
ping -D 8.8.8.8    # Linux/macOS
ping 8.8.8.8 -f    # Windows
```

  - `-D` (Linux/macOS) and `-f` (Windows) activate the `DF` flag.

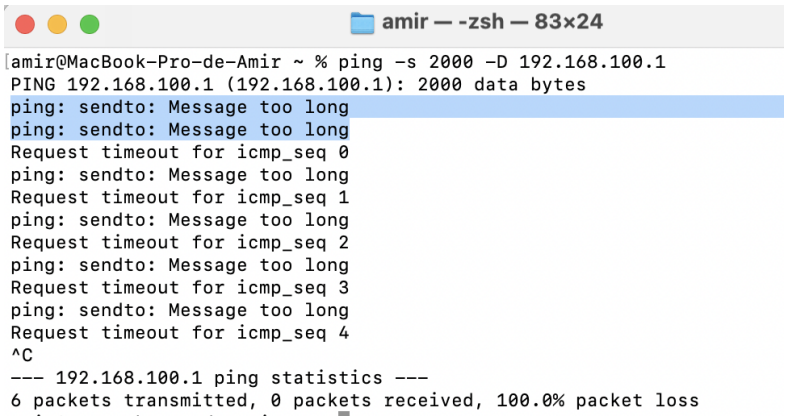

Figure 9: Ping result with DF activated

- Observe the ICMP echo in Wireshark :
  - If the packet size is greater than the MTU, the frame will be dropped by the local interface before sending it, figure 9.
  - If the packet size is less than the MTU, an ICMP packet will be generated with the DF activated, figure 10.
- We can filter by destination IP addresse and the ICMP messages

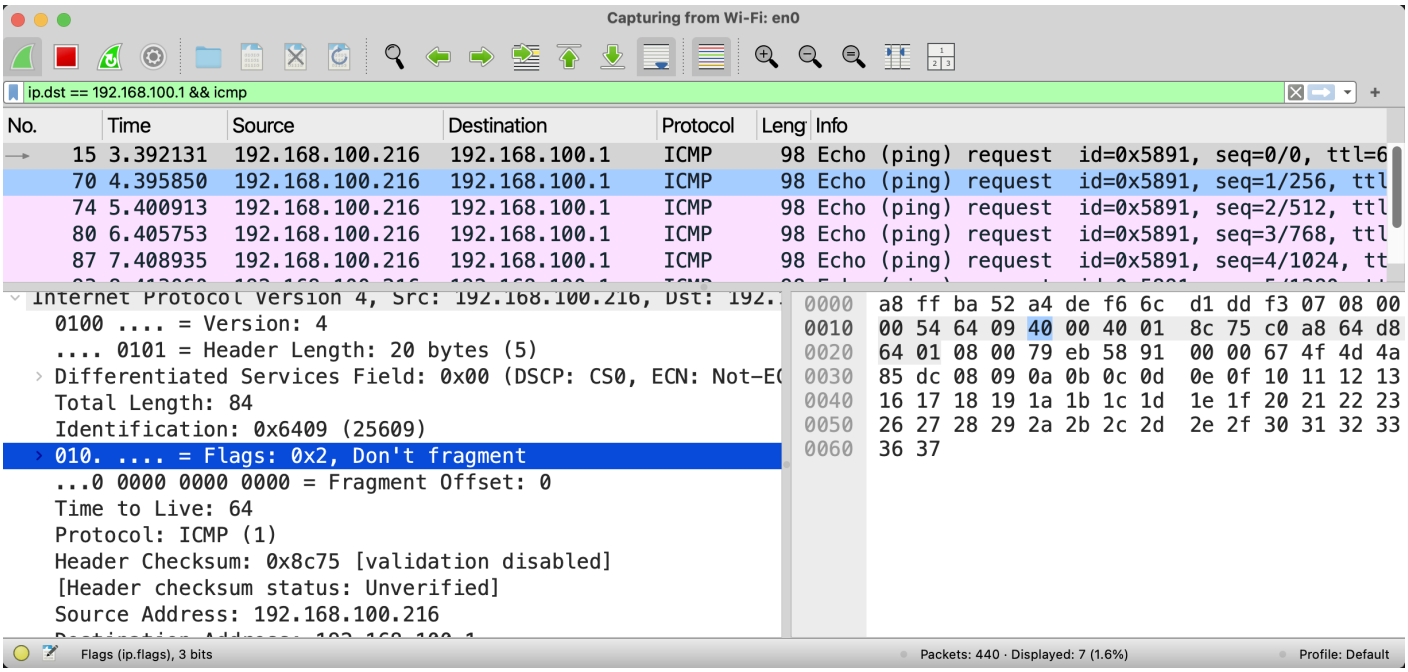

Figure 10: Fragmented ICMP message captured with Wireshark

Exploring the ICMP protocol with Scapy

Task 1: Simple ICMP requests and responses
Step 1 : Script Scapy: Sending an Echo request (Ping)

- Create a Python file named `icmp_ping.py` with the following contents:

```python
from scapy.all import *

# Building an ICMP Echo Request packet

icmp_request = IP(dst="8.8.8.8") / ICMP(type=8) / "Hello, this is a test"

# Send the packet and receive the reply
response = sr1(icmp_request, timeout=2)

# Check answer
if response:
    print("Response received")
    response.show()
else:
    print("No response from the server.")
```

Step 2 : Packet analysis in Wireshark

- Use the filter `ip.dst == 8.8.8.8 && icmp` to capture the packet, figure 11

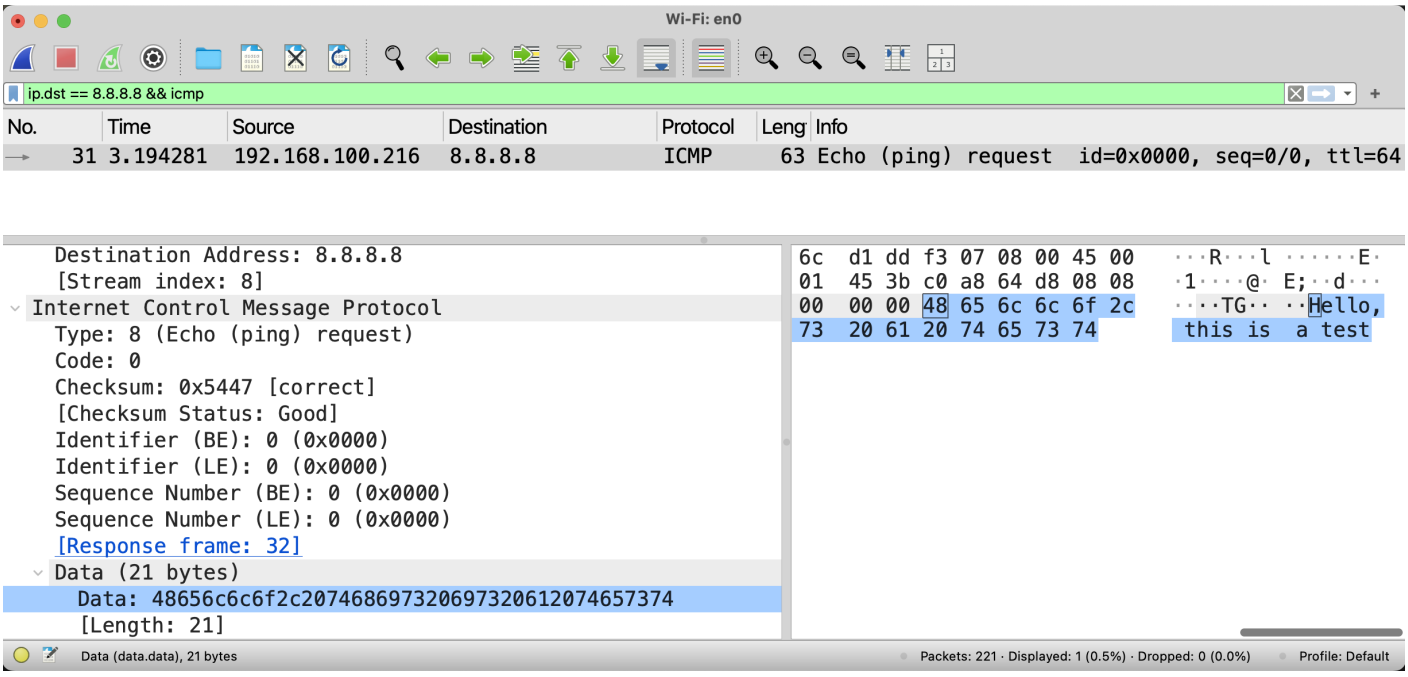

Figure 11: Capturing the ICMP packet sent by the script

- Launch the script `icmp_ping.py` and analyse the ICMP requests (Type 8, Code 0) and responses (Type 0, Code 0).

Task 2: Simulate an ICMP error
Step 1: Generate an ICMP Destination Unreachable packet

- Add the following code to a Python file named `icmp_error.py` :

```python
from scapy.all import *

# Simulate an ICMP Destination Unreachable message
ip_header = IP(src="192.168.1.100", dst="8.8.8.8")
udp_segment = UDP(sport=1234, dport=80)
icmp_error = IP(src="8.8.8.8", dst="192.168.1.100") / ICMP(type=3, code=3) / ip_header /
    udp_segment

# Send ICMP packet
send(icmp_error)

print("ICMP Destination Unreachable message sent.")
```

- We will use the filter `icmp.type == 3` in order to display the message after lauching the script `icmp_error.py`

Extra examples

Example 1: Scanning a network

You need to develop an advanced ICMP scanner to detect all active hosts on a subnet (e.g. 192.168.100.0/24), you need to update the network address according to yours.

- Create a Python file `icmp_network_scan.py` :

```python
from scapy.all import *

def scan_network(network):
    print(f"Scanning the network: {network}")
    # Generate ICMP Echo Request packets for each IP address on the network
    ans, unans = sr(IP(dst=network)/ICMP(type=8), timeout=2, verbose=0)

    # Process responses received
    print("Hosts responding to ICMP Echo Request:")
    for sent, received in ans:
        print(f"{received.src} is alive.")

    # Non-responding hosts
    if unans:
        print("\nHosts not responding:")
        for sent in unans:
            print(sent.dst)

# Scan a complete network
if __name__ == "__main__":
    scan_network("192.168.100.0/24")
```

Example 2: Personalized ping

Here is an example of interaction between a sender and a receiver using Scapy to illustrate a personalised ICMP exchange. The aim is to simulate a scenario where a sender sends a request, and a receiver responds with a modified or customised response.

- **The sender** sends an ICMP Echo Request with a personalised message.
- **The receiver** listens to incoming ICMP packets and responds with an ICMP Echo Reply containing a different message.

Code for the sender (icmp_sender.py)

```python
from scapy.all import *

def send_ping(target_ip):
    message = "Hello from sender!"
    icmp_request = IP(dst=target_ip) / ICMP(type=8) / message

    print(f"Sending ICMP Echo Request to {target_ip}")
    response = sr1(icmp_request, timeout=2)

    if response:
        print("Received response:")
        print(response.show())
    else:
        print("No response received.")

if __name__ == "__main__":
    target = "192.168.100.100"  # Receiver IP address
    send_ping(target)
```

Code for the receiver (icmp_receiver.py)

```python
from scapy.all import *

def handle_packet(packet):
    if ICMP in packet and packet[ICMP].type == 8:  # Checks whether it is an ICMP Echo request
        print("ICMP Echo Request received:")
        packet.show()

        # Build an ICMP Echo Reply
        reply = IP(src=packet[IP].dst, dst=packet[IP].src) / ICMP(type=0) / "Hello from receiver!"
        print("Sending ICMP Echo Reply...")
        send(reply, verbose=0)

if __name__ == "__main__":
    print("Listening for ICMP Echo Requests...")
    sniff(filter="icmp", prn=handle_packet)
```